

TRUSTLINE

Code-Derived Architecture & Security Guide

A technical guide for hackathon judges, mentors, recruiters, and developers

Document attribute	Value
Scope	Current repository implementation only — frontend, backend, migrations, runtime configuration, tests, and demo screens.
Architecture	React/Vite SPA → Express API → PostgreSQL; browser WebAuthn ceremony; Node.js cryptography.
Status language	IMPLEMENTED means code executes; DEMO means a visual/local simulation; NOT IMPLEMENTED identifies an intentional current boundary.
Produced	Architecture review of the repository supplied for this request.

Reading note This guide avoids treating design intent as delivered functionality. Where the UI demonstrates a concept without a backing service, it is labelled DEMO.

Contents

1. Executive Summary
2. High-Level Architecture
3. Repository Structure
4. Screen-by-Screen Walkthrough
5. Authentication & Session Design
6. TOTP Implementation
7. Approval Workflow, Signatures & Receipts
8. Demo Features & Delivery Status
9. Database Model
10. Libraries & Runtime Dependencies
11. File Dependency Maps
12. Security Architecture & Current Boundaries
13. Complete User Journey
14. Developer Guide
15. Quick Reference

1. Executive Summary

TrustLine is a passkey-first approval prototype for demonstrating higher-assurance access, adaptive step-up authentication, policy-backed approvals, cryptographic vote receipts, and tamper-evident audit records. It is built as a browser SPA with a TypeScript/Express API and PostgreSQL persistence.

The project addresses a familiar enterprise risk: a sensitive action should not rest on a password or a single opaque approval. Its implementation combines WebAuthn authenticators, TOTP step-up, short-lived access tokens, rotated refresh tokens, approval policies, Ed25519 signatures, and a SHA-256-linked audit trail.

What differentiates it The implementation connects identity evidence to an approval record: successful passkey registration creates a separate Ed25519 signing key; a vote is signed server-side; and the receipt endpoint returns votes, public keys, and related audit entries.

2. High-Level Architecture

Runtime architecture



Execution flow — arrows denote the runtime hand-off.

Layer	Actual responsibility
Browser	Vite-served React routes, WebAuthn browser ceremonies, QR rendering, sessionStorage token holder, and demo visualizations.
Frontend	`apiClient` performs JSON fetches; authenticated helpers attach Bearer tokens. React Router selects every screen.
Backend	Express parses JSON, applies CORS / Pino logging, mounts auth, approval, and ledger routers, and exposes `/health`.
Database	PostgreSQL stores users, credentials, token hashes, policies, requests/votes, delegations, signing keys, login events, and audit entries.
Authentication	SimpleWebAuthn verifies origin/RP ID/challenge and updates credential counters. JWT middleware protects API endpoints.
Security	Node crypto supplies HMAC, SHA-256, AES-GCM, Ed25519, random values, and timing-safe TOTP comparison.

Deployment note: `docker-compose.yml` starts PostgreSQL, Redis, backend, and nginx frontend. Redis and `REDIS\_URL` are present in Compose, but the tracked runtime imports no Redis client; challenge and step-up rate-limit state are process-memory Maps.

3. Repository Structure

Location	Purpose & important files
frontend/src/pages	Route-level screens: `RegisterPage`, `LoginPage`, `DashboardPage`, and three demonstration pages.
frontend/src/components	Reusable UI primitives and demo-only views: `PushSimulator`, `AdaptiveTrustEngine`, `TrustTimeline`, `JudgeMode`, `AttackSimulation`.
frontend/src/lib	`apiClient.ts` centralizes fetch/error behavior; `auth.ts` manages sessionStorage tokens.
frontend/src/App.tsx	BrowserRouter route ownership; `/` redirects to `/login`.
backend/src/routes	HTTP controllers for auth, approvals, and receipt retrieval.
backend/src/services	Persistence, WebAuthn, TOTP, risk, session, policy, quorum, signing, audit, keys, and delegation logic.
backend/src/middleware	`requireAuth` verifies Bearer JWT and attaches `req.userId`; `errorHandler` produces JSON errors.
backend/migrations	Ordered node-pg-migrate schema source of truth, `00001` through `00011`.
backend/src/scripts & tests	Seed/smoke/load/chain verification utilities and Vitest integration/service tests.
docs & root scripts	Demo and presentation support material plus `start-demo.sh`; not used as implementation evidence.

4. Screen-by-Screen Walkthrough

Every browser route is declared in ``frontend/src/App.tsx``. The table below distinguishes network-backed screens from local visual demonstrations.

Screen / component	Purpose & state	API / backend path	Buttons and downstream effect
<code>`/register`</code> — RegisterPage	Email, phase (<code>`form`</code> / <code>`totp-qr`</code> / <code>`done`</code>), QR data URL and 6-digit code. Uses <code>`@simplewebauthn/browser`</code> and <code>`qrcode`</code> .	POST register/options → register/verify → totp/setup → totp/verify. <code>`auth.routes`</code> → <code>`webauthn.service`</code> / <code>`totp.service`</code> → users, credentials, signing_keys.	Register passkey begins browser ceremony. Confirm & finish verifies TOTP then redirects to login.
<code>`/login`</code> — LoginPage	Email, phase (<code>`email`</code> / <code>`number-match`</code> / <code>`totp`</code>), pending token, code. Uses a local PushSimulator.	POST login/options → login/verify; conditional login/step-up. <code>`webauthn.service`</code> , <code>`risk.service`</code> , <code>`session.service`</code> ; users/credentials/login_events/refresh_tokens/audit_log.	Continue with passkey starts browser assertion. Number match advances only locally. Submit code calls step-up and stores tokens.
<code>`/dashboard`</code> — DashboardPage	Authenticated dashboard state: pending requests, sessions/activity, a seeded/demo action form and vote controls.	Authenticated approval and auth endpoints via <code>`authenticatedGet`</code> / <code>`authenticatedPost`</code> / <code>`authenticatedDel`</code> ; approval routes, session routes, ledger receipt link.	Create request, approve/deny, break-glass, session revoke, refresh, and navigate to demo pages. Each API action sends Bearer access token.
<code>`/demo/dispute`</code> — DisputeDemoPage	Selects a resolved request and renders its receipt / audit information.	GET approval requests and GET ledger/receipt/:requestId; <code>`ledger.routes`</code> → <code>`audit.service.getReceiptForRequest`</code> .	Load/choose request retrieves requester-scoped votes, public keys, and audit rows.
<code>`/demo/attack`</code> — AttackDemoPage	Attack/replay and rate-limit demonstration view.	Mixed: visual simulation plus login step-up behavior when driven by the app.	UI controls exercise narrative/demo state; not a general penetration-testing endpoint.
<code>`/demo/phishing-clone`</code> — PhishingCloneDemoPage	Origin-binding awareness demonstration for passkeys.	No production backend transaction required for the clone visualization.	Demonstrates that a WebAuthn ceremony is bound to expected origin/RP ID.

Registration click path



Execution flow — arrows denote the runtime hand-off.

Vote click path



Execution flow — arrows denote the runtime hand-off.

Supporting components: ``ui.tsx`` provides wordmark, icon, spinner and alert primitives. ``PushSimulator`` is local UI state. ``AdaptiveTrustEngine``, ``TrustTimeline``, ``JudgeMode``, and ``AttackSimulation`` use explicit mock/demo data and do not call the backend.

5. Authentication & Session Design

Registration is passwordless. ``generateRegistrationOptionsForUser`` upserts a user, loads credential IDs to exclude, creates SimpleWebAuthn registration options, and stores the challenge by email in memory. ``verifyRegistration`` verifies challenge, origin and RP ID; stores the credential public key and counter; creates an Ed25519 signing keypair; then deletes the challenge.

Element	Code-derived implementation
Passkey credential	Only credential ID, WebAuthn public key (<code>`bytea`</code>) and sign counter are stored. Authenticator private keys never enter TrustLine.
Challenge	Registration and login use separate in-memory <code>`Map<string,string>`</code> stores keyed by email; successful verification deletes the challenge.
Login & counter	<code>`verifyAuthenticationResponse`</code> checks expected challenge/origin/RP ID and writes <code>`newCounter`</code> to prevent authenticator assertion replay.
Risk decision	No prior login history → high; new IP with known user agent → medium; otherwise low. It writes a fail-open risk audit event.
Step-up	Medium/high returns a 5-minute JWT <code>`{sub,purpose:'step_up',ip,userAgent}`</code> . TOTP is checked after a per-user, in-memory >3 attempts/60 seconds limiter.
Access JWT	<code>`jsonwebtoken.sign({sub:userId})`</code> with 15-minute expiry; <code>`requireAuth`</code> verifies signature/expiry and assigns <code>`req.userId`</code> .
Refresh token	40 random bytes hex; only SHA-256 hash stored; seven-day expiry. Rotation revokes the consumed row; reuse revokes the entire family.
Browser storage	Access and refresh tokens are maintained in module variables and <code>`window.sessionStorage`</code> , ending with the browser session.

Current boundary Registration/TOTP setup accepts a ``userId`` in the body without an authenticated caller. Secrets are stored in the users row; encryption-at-rest and authorization hardening are not implemented.

6. TOTP Implementation

The TOTP service is a custom Node `crypto` implementation; no OTP library is listed in the backend dependencies. QR image generation is frontend-only (`qrcode`) after the backend returns an `otpauth://` URI.

Function / concern	Actual behavior
Secret generation	<code>randomBytes(20)</code> produces 160 bits and <code>encodeBase32</code> makes unpadded RFC 4648 Base32.
Base32 parsing	<code>decodeBase32</code> normalizes whitespace/hyphens/case, validates alphabet and valid padding/length, then converts 5-bit symbols to bytes.
HOTP	<code>generateHOTP</code> uses <code>createHmac('sha1', secret)</code> , eight-byte big-endian counter, RFC 4226 dynamic truncation, and six digits.
TOTP	<code>generateTOTP</code> computes <code>floor(timestampMs / 30,000)</code> and delegates to HOTP.
Verification	<code>verifyTOTP</code> checks <code>-1..+1</code> time steps by default and uses <code>timingSafeEqual</code> for each candidate without early loop exit.
Provisioning	<code>buildOtpAuthUri</code> emits issuer TrustLine, SHA1, 6 digits, 30-second period. RegisterPage uses <code>QRCode.toDataURL</code> .
Persistence	<code>generateSecretForUser</code> updates <code>users.totp_secret</code> and disables; <code>verifyCode</code> validates then sets <code>totp_enabled=true</code> .

TOTP enrollment



Execution flow — arrows denote the runtime hand-off.

Why SHA-1 here: it is the interoperable HOTP/TOTP PRF selected by the current provisioning URI and common authenticator defaults. SHA-1 is not used as the project’s general-purpose integrity hash; the audit and token hashing use SHA-256.

7. Approval Workflow, Signatures & Receipts

Area	Implemented behavior
Policy	Authenticated callers can create/list/get `approval_policies`, with quorum type, N/M values, eligible roles, escalation fields, geo fence and a version.
Request	`createRequest` snapshots the policy version and inserts a pending `approval_requests` row with JSON action payload.
Vote	`submitVote` finds an owned pending request, assesses delegation state, signs `{requestId, decision, timestamp}`, inserts one vote per approver, evaluates quorum, and may resolve status.
Quorum	Pure `evaluateQuorum` handles `n_of_m`, `single_senior`, and `role_weighted`; role-weighted currently resolves by count semantics.
Escalation	A job runs every 30 seconds and flags eligible pending requests `escalated=true`.
Break-glass	Owner-scoped force approval marks `break_glass=true`, `needs_review=true` and resolves. It does not append an audit entry.
Signature	Ed25519 private key is decrypted, canonical JSON is signed, and a base64 signature is stored with the vote. Public key accompanies receipt votes.
Receipt	Requester-scoped ledger endpoint returns request ID, votes with public keys, and audit entries whose payload contains request ID.

Authorization reality The current demo does not implement reviewer assignment or roles. Pending and vote queries scope a request to its requester, so the owner can vote on their own request; `eligible\_roles` is not a complete RBAC enforcement system.

Audit chain: `appendAuditEntry` begins a transaction, locks the latest row, derives `this\_hash = SHA-256(prev\_hash + JSON.stringify(payload))`, inserts, and commits. The first row uses the conceptual `GENESIS` sentinel while the stored `prev\_hash` is null.

8. Demo Features & Delivery Status

Feature	Status	Evidence / interpretation
WebAuthn registration & login	IMPLEMENTED	SimpleWebAuthn server/browser ceremonies, credential persistence, counter updates, origin/RP checks.
TOTP enrollment and step-up	IMPLEMENTED	Custom crypto service, QR display, code verification, in-memory attempt limiter.
Local number matching	DEMO	`PushSimulator` presents a client-side phase before backend TOTP; no push provider or server-side challenge.
Adaptive Trust Engine / Trust Timeline	DEMO	Explicit mock data, visual explanation, no backend calls.
Attack simulation	DEMO	Visualization/narrative around replay and rate limit, not an external attack-testing subsystem.
Phishing clone screen	DEMO	Origin-binding education screen; real WebAuthn expected origin/RP checks are separately implemented.
Policy / approval persistence	IMPLEMENTED	Routes, services, migrations, votes, quorum and escalation flag.
Cryptographic vote receipt	IMPLEMENTED	Ed25519 signatures, public-key receipt data, hash-chain audit records.
Redis-backed shared state	NOT IMPLEMENTED	Compose runs Redis, but source uses in-memory maps and no Redis import.
Multi-reviewer role/RBAC model	NOT IMPLEMENTED	No user roles/reviewer assignment and owner-scoped vote behavior.
Secret encryption for TOTP	NOT IMPLEMENTED	TOTP secret field is text; signing private keys are encrypted separately.

9. Database Model

Table	Purpose, relationships and indexes
users	Identity row: UUID, unique email, display name, timestamps; later migration adds `totp_secret` and `totp_enabled`.
webauthn_credentials	Many credentials per user. Stores unique credential ID, public key and counter; indexed by `user_id`; cascades on user delete.
refresh_tokens	Session rotation records. Unique token hash, family, revoked, expiry; indexes on user, family and token hash.
approval_policies	Policy definitions; name indexed. JSONB eligible roles/escalation/geo fence and integer version.
approval_requests	Policy and requester references, policy version snapshot, action JSONB, status/timestamps, escalation/break-glass/review flags; indexes policy, requester, status and status+escalated.
approval_votes	Request/approver references, decision, signature; unique `(request_id, approver_id)` and indexes on both foreign keys.
audit_log	Append-only-by-service hash chain: bigserial ID, type, payload JSONB, previous/current hash, timestamp; indexes created_at and entry_type.
delegations	Delegator/delegate user references and expiry; indexes each identity column.
signing_keys	One-to-one user key pair: public PEM and AES-GCM encrypted private PEM; cascades on user deletion.
login_events	User, optional IP/user agent and time; indexes user and created_at for risk scoring.

Key data relationships



Execution flow — arrows denote the runtime hand-off.

Write behavior is concentrated in services. Inserts create users/credentials/tokens/policies/requests/votes/delegations/audit entries; updates refresh revocation, WebAuthn counter, TOTP enablement, request resolution/escalation and break-glass flags. Deletion is principally foreign-key cascade; the visible session revoke API marks token families revoked rather than deleting them.

10. Libraries & Runtime Dependencies

Library / runtime	Purpose	Where used / why
React, React Router, Vite	SPA rendering and route selection	Frontend `main.tsx`, `App.tsx`, page components; lightweight client application structure.
@simplewebauthn/browser	Browser WebAuthn ceremonies	Register/Login pages call `startRegistration` / `startAuthentication`.
@simplewebauthn/server	WebAuthn option generation and verification	`webauthn.service.ts`; validates authenticators against challenge/origin/RP ID.
Express, cors, pino/pino-http	HTTP API, controlled CORS, logging	`app.ts`, routes, server middleware.
pg, node-pg-migrate	PostgreSQL access and schema migrations	Pool/service queries and ordered migration scripts.
jsonwebtoken	Access, pending step-up tokens	`session.service`, auth route, `requireAuth`.
Node crypto	Randomness, HMAC, SHA-256, AES-GCM, Ed25519	TOTP, sessions, key encryption, signing, audit chain.
qrcode	QR data URL renderin	RegisterPage transforms provisioning URI into an image.
Tailwind CSS	UI styling	Frontend class usage / Tailwind config.
Vitest, Supertest	Tests	Service and integration tests in backend tests/services.

11. File Dependency Maps

Registration



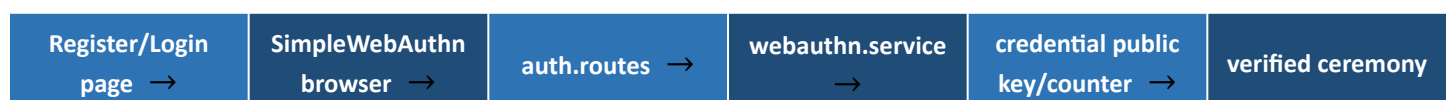
Execution flow — arrows denote the runtime hand-off.

Login / session



Execution flow — arrows denote the runtime hand-off.

Passkey



Execution flow — arrows denote the runtime hand-off.

TOTP



Execution flow — arrows denote the runtime hand-off.

Dashboard approval



Execution flow — arrows denote the runtime hand-off.

Receipt / audit



Execution flow — arrows denote the runtime hand-off.

12. Security Architecture & Current Boundaries

Control	Why it is used in the current design
Passkeys / WebAuthn	Authenticator private keys remain on device; WebAuthn validates RP ID and page origin, reducing credential phishing compared with passwords.
TOTP	Independent knowledge/possession-style code used for risk-driven step-up and enrollment confirmation.
JWT	Compact signed access token permits middleware authorization using a subject and an expiry.
Refresh rotation	Protects long-lived session continuity by storing only SHA-256 hashes, revoking consumed tokens and detecting family reuse.
SHA-256	Used for refresh-token hashing, signing-key encryption-key derivation and audit-chain linkage.
AES-256-GCM	Encrypts per-user Ed25519 private PEMs with random 96-bit IV and authenticated tag; supports previous secret migration.
Ed25519	Signs canonical vote payloads; receipts provide public key plus signature for independent verification.
Audit chain	Serializable transaction plus prior hash makes alteration/deletion evident during chain verification.

Production hardening priorities Move challenge/rate-limit state to shared durable storage; encrypt TOTP secrets; bind TOTP setup to an authenticated or one-time enrollment authorization; add RBAC/reviewer assignment and separation of duties; append break-glass audit events; add CSRF/session policy and external key-management controls.

13. Complete User Journey

End-to-end journey



Execution flow — arrows denote the runtime hand-off.

Stage	What actually happens
Opening	React Router sends `/` to `/login`. The user can reach registration with the account link.
Registration	Email → options endpoint → platform authenticator → verification. A public credential and Ed25519 keypair are persisted.
Authenticator	The backend produces a standard otpauth URI; the UI turns it into QR; first valid code enables TOTP.
Login	The browser signs an authentication assertion. Server verification checks the expected origin/RP ID/challenge and updates counter.
Adaptive trust	Heuristic risk selects low, medium or high. Medium/high return a temporary pending token for TOTP step-up.
Dashboard	The SPA attaches its sessionStorage access token to authenticated API calls. It reads and acts on owner-scoped requests.
Approval	A request uses a policy snapshot. A vote is signed with server-held encrypted Ed25519 private key, stored, and evaluated by quorum.
Receipt & audit	The user can retrieve request vote signatures/public keys and matching audit entries. Audit linkage can be independently checked by the bundled verifier script.

14. Developer Guide

Start with the boundaries, then follow an endpoint end-to-end. `frontend/src/App.tsx` names every screen; each page reveals its API calls. `backend/src/app.ts` mounts the routers. Routes provide request validation and service calls; services are the business/persistence source; migrations define stored data.

If you need to change...	Start here
Passkey ceremony	<code>`backend/src/services/webauthn.service.ts`</code> , then <code>`auth.routes.ts`</code> , then Register/Login pages.
TOTP behavior	<code>`backend/src/services/totp.service.ts`</code> and tests, then Register/Login pages.
Session or token policy	<code>`session.service.ts`</code> , <code>`requireAuth.ts`</code> , token helpers in frontend <code>`lib/auth.ts`</code> .
Approval mechanics	<code>`approval.routes.ts`</code> , <code>`request.service.ts`</code> , <code>`quorum.service.ts`</code> , <code>`policy.service.ts`</code> , migrations.
Receipt / proof	<code>`signing.service.ts`</code> , <code>`keys.service.ts`</code> , <code>`audit.service.ts`</code> , ledger route, DisputeDemoPage.
UI / API integration	<code>`apiClient.ts`</code> is the sole fetch wrapper; authenticated requests use <code>`authedGet/Post/Del`</code> .
Schema	Create a new ordered migration; do not treat TypeScript interfaces as schema.
Verification	Run backend tests; use smoke/seed/verifyChain scripts against a configured database.

15. Quick Reference

Topic	Cheat sheet
Authentication	Registration: <code>`/api/auth/register/options`</code> + <code>`/verify`</code> ; Login: <code>`/login/options`</code> + <code>`/verify`</code> ; step-up: <code>`/login/step-up`</code> ; access JWT: 15m; refresh token: 7d hashed and rotated.
Frontend	Routes in <code>`App.tsx`</code> ; fetch in <code>`lib/apiClient.ts`</code> ; tokens in <code>`lib/auth.ts`</code> ; screens in <code>`pages/`</code> ; demo-only visuals in components.
Backend	<code>`app.ts`</code> mounts <code>`/api/auth`</code> , <code>`/api/approval`</code> , <code>`/api/ledger`</code> ; <code>`requireAuth`</code> gates protected calls; services own database/crypto decisions.
Data	Users → credentials / refresh tokens / login events / signing keys. Policies → requests → votes. Audit records link by prior hash and requestId payload.
Caveat	Treat local in-memory Maps, owner voting, non-encrypted TOTP secrets, and non-audited break-glass as prototype boundaries, not production assurances.

End